

EMS Data Analysis and Forecasting

For CSIS 4495 Applied Research, Section 002

In Partnership with Riipen
Tony Tsui, Solaris Canada

A Midterm Report Submitted to
Douglas College
New Westminster, BC, Canada

by
Dundee Adriatico
300393449
April 8, 2026

GitHub Repo | https://github.com/dondemici/W26_4495_S2_DundeeA

Contents

Introduction	3
Summary of the Research Project	4
Changes to Proposal	5
Project Completion Timeline.....	9
Overview.....	9
Phase Breakdown & Milestones	9
Gantt Chart.....	12
Implemented Features	13
Reflections	15
AI Use Section	15
Work Log	18
Concluding Remarks	21
References	23
Journals and Articles.....	23
Web Reports / Government and Organizational Reports	23
Theses / Dissertations.....	24
Appendix A: Installation Guide	25
Appendix B: User Guide	25
Appendix C: List of Prompts	25

Introduction

The Emergency Medical Services (EMS) play a critical role in the healthcare system by providing rapid, pre-hospital care in unpredictable and often hazardous environments (Reichard et al., 2017; Maguire et al., 2023). EMS clinicians routinely lift and move patients, work in cramped or unsafe scenes, and face time pressure and environmental stressors. These conditions expose them to a wide range of occupational hazards, including musculoskeletal strain, blood and body-fluid exposures, violence, and transportation incidents (Reichard et al., 2017; Marsh et al., 2025). Multiple studies have shown that EMS workers experience substantially higher rates of work-related injuries and exposures than the general workforce and even other healthcare workers (Reichard et al., 2017; Maguire et al., 2023; Xia & Collie, 2020).

At the same time, EMS systems must manage fluctuating call volumes, seasonal patterns, and special events that create workload peaks. When staffing and resources are not aligned with these peaks, individual clinicians can experience high cognitive and physical load, fatigue, and rushed task performance. Prior research in both EMS and hospital settings suggests that higher workload and fatigue are associated with increased risk of injuries, musculoskeletal disorders, and safety incidents (Alhazmi et al., 2024; Hammad et al., 2024; Reichard et al., 2017). However, most EMS agencies still monitor injury rates retrospectively and rely on simple descriptive statistics or lagging indicators. They often discover “bad weeks” only after a cluster of exposure or injury reports has already occurred (Reichard et al., 2017; National EMS Information System [NEMSIS], 2025).

This project focuses on one component of this broader safety challenge: providing EMS leaders with a simple, interpretable application that summarizes recent work-related exposure events and evaluates the accuracy of their existing operational forecasts. Rather than developing complex machine-learning models, the project uses routinely collected EMS data and straightforward forecasting approaches that can be implemented and understood in practice. The central questions are:

- Can routinely available EMS data be transformed into clear visualizations and summary metrics that highlight recent patterns in exposure events?
- Are there temporal patterns (e.g., trends or seasonality) in exposure events that can inform proactive staffing and safety planning (Qian et al., 2016; Tarisayi et al., 2024)?
- How accurate are simple exposure forecasts over short time horizons, and how can forecast errors be communicated to EMS managers in a way that supports decision making?

Answering these questions is important because even a modestly accurate, transparent forecast—paired with an intuitive app interface—could enable EMS leaders to plan staffing, training, and

safety interventions before high-risk periods, rather than reacting only after injuries and exposures have already occurred. In addition, the EMS App integrates an AI-powered Planning Assistant (PAi) that translates exposure and forecast results into concrete recommendations (for example, suggested staffing adjustments or focused safety messaging), making the analytics directly actionable for EMS managers.

These questions are addressed through the EMS App, a web-based dashboard built with tech stack, such as Python and SQL, that ingests routinely collected EMS data and generates short-term exposure forecasts using different models like moving averages, exponential smoothing, Prophet, Sarima, Holt and Holt-Winters. The app is designed for EMS managers, emphasizing interpretability, transparency of forecast accuracy, and ease of integration into existing workflows.

Summary of the Research Project

This project developed a prototype EMS Work-Related Injury Forecast web application that helps EMS leaders monitor weekly exposure events and anticipate near-term risk using routinely collected operational data. The app ingests historical EMS exposure records from a fact table exported to CSV, aggregates them into a cleaned daily series, and then resamples to weekly counts suitable for forecasting. It focuses on being transparent and operationally useful rather than on complex black-box models, aligning with current recommendations that decision tools for emergency care remain interpretable and easy to integrate into existing workflows.

The EMS Forecaster is implemented as a Streamlit application with a SQL Server-compatible data pipeline, structured around a single weekly time series of exposure counts. The data-processing pipeline parses raw timestamp strings, filters valid events, constructs a continuous daily series, and then collapses this to Monday-based weekly totals for visualization and modeling. The app also includes optional enrichment with external context via live calls to the Open-Meteo API for weather variables (rain, snow, temperature) and the Ticketmaster Discovery API for major events in the Vancouver area, allowing future versions to explore how environmental conditions and special events may relate to exposure risk.

For forecasting, the app exposes a menu of simple, interpretable time-series models rather than a single complex machine-learning pipeline. Available methods include naive and moving-average baselines, simple exponential smoothing, Holt and Holt-Winters exponential smoothing, and a seasonal ARIMA (SARIMA) specification with weekly seasonality; a small experimental random-forest model is also available but clearly labeled as exploratory. Users select a model and horizon (1–24 weeks), after which the app generates a weekly forecast with approximate upper and lower bounds and overlays it with the historical series in an interactive line chart. This design

mirrors other Streamlit-based forecasting tools that emphasize visual comparison between history and forecast for non-technical stakeholders.

A core addition in the final version of the project is an explicit focus on forecast accuracy and reliability. Instead of only producing forward-looking predictions, the app performs a rolling backtest that uses the most recent weeks as a test window, fits the selected model on earlier history, and evaluates performance using standard metrics such as mean absolute error (MAE), root mean squared error (RMSE), and mean absolute percentage error (MAPE), consistent with the EMS forecasting literature. These metrics are displayed alongside the forecast as simple numerical tiles, and the app computes MAE as a percentage of the average weekly exposure level to help managers interpret error magnitudes in familiar units. This accuracy-feedback loop is intended to discourage over-confidence in any single model and to make the limitations of the forecasts explicit.

Finally, the EMS Forecaster incorporates an AI-powered Planning Assistant (PAi) to turn raw forecasts into actionable recommendations. Using Google's Gemini API, PAi receives a structured prompt that includes a summary of the recent 8-week average, the next-4-week forecast, and the backtest accuracy metrics for the chosen model. It then generates a short, plain-language narrative (4–6 sentences) aimed at EMS managers, emphasizing staffing, training, and PPE planning, and explicitly commenting on forecast confidence. This moves the app from a passive analytics dashboard toward a lightweight decision-support tool, reflecting broader trends in using AI assistants to translate operational analytics into concrete next steps in healthcare operations.

Changes to Proposal

Since submitting the initial proposal, the project has evolved in several important ways to better match the realities of the NEMSIS research data, local data-quality constraints, and the time available in the term. The scope has narrowed from broad demand and injury prediction to a focused goal of forecasting weekly work-related exposure events for a single year, using interpretable time-series models and explicit forecast-accuracy checks instead of a large comparison of black-box ML models. The dashboard has been refined into a lightweight EMS exposure-risk forecaster with a built-in AI Planning Assistant (PAi) that translates the forecasts and backtest metrics into plain-language recommendations for managers. The technical stack has also shifted toward a pragmatic combination of SQL Server for preprocessing, CSV exports for the final dataset, and a Streamlit app for visualization, forecasting, and AI-generated guidance.

Change ID	What changed	Details of the change	Justification / reasons
a) Scope	From “forecast demand + predict disabling injuries” to	The project now focuses on forecasting weekly counts of EMS work related	The NEMSIS research dataset and local extracts reliably support a binary exposure flag but not

Change ID	What changed	Details of the change	Justification / reasons
	“forecast weekly work related exposure events in one year”	exposure events using a single cleaned 2024 cohort, instead of jointly modeling demand peaks and multiple injury types across several years.	consistent, detailed injury mechanism or severity fields. Trying to model disabling injuries and multi year demand within an 8 week project window proved overly ambitious. Narrowing to exposure flagged events in one year allowed a complete, end to end pipeline to be delivered (data engineering, forecasting, accuracy checks, and UI).
b) Outcomes & joins	From assuming rich hospital outcome linkages to using the exposure flag as the primary safety outcome	Early attempts to join incidents to hospital outcome elements showed extensive nulls and Not Applicable codes in key fields, making it difficult to define “disabling” injuries in a reproducible way. The project now treats the work related exposure flag as the sole safety outcome for forecasting.	The public research data and local extracts do not contain sufficiently complete or reliable hospital outcome detail to support a robust severity definition. Continuing to pursue complex joins around poorly populated outcome fields would have consumed time without adding actionable signal. Using the exposure flag directly keeps the outcome simple, well defined, and consistently available.
c) Time base	From multiple years of incident/call times to a single year FACT/PCR based time series exported to CSV	The original plan used multiple years of call/incident timestamps. In practice, work with the 2024 exposure positive cohort showed that FACT/PCR related times were more consistently populated for exposure events and easier to process. The final app reads from a pre aggregated CSV of exposure records and builds a daily then weekly series from that.	Differences in completeness across NEMSIS time fields, combined with local data quality issues and the need to keep compute simple, made a multi year, multi timestamp strategy brittle. Anchoring the series in one year with a consistently populated time variable and exporting to CSV reduced complexity, eliminated heavy joins at runtime, and provided a stable weekly series for the Streamlit app.
d) Modeling strategy	From broad ML comparison to interpretable time	The initial proposal emphasized comparing generic ML models	Once the outcome was defined as a univariate weekly exposure count for a single year, the

Change ID	What changed	Details of the change	Justification / reasons
	series models plus an experimental ML option	(regression, random forests, gradient boosting) for demand and injury prediction. The final app instead centres on transparent time series methods (naive and moving average baselines, exponential smoothing variants, and SARIMA), with a clearly labeled “ML (Experimental)” random forest option rather than a full ML benchmarking study.	problem was better matched to classical time series forecasting than to high dimensional tabular ML. Focusing on a small family of interpretable models supported clear explanations, visualizations, and backtesting within the available time. Retaining one experimental ML model acknowledges modern methods without over promising a full ML pipeline.
e) Dashboard scope	From broad QA automation and multi injury “hot spots” to an exposure risk forecasting dashboard with accuracy feedback	The initial dashboard concept included multiple injury types, demand views, and geospatial “hot spot” analyses. The final product is a streamlined dashboard that shows historical weekly exposure counts, a configurable forecast for the next 1–24 weeks, prediction bands, a backtest accuracy panel, and a simple “original vs updated” forecast illustration.	Data limitations on detailed injury mechanisms and hospital outcomes, combined with time constraints, made a broad QA engine unrealistic. Focusing the UI on exposure risk forecasting and accuracy metrics produced a coherent tool that EMS leaders can understand and act on, while still fitting into the project’s time and data constraints.
f) Forecast accuracy and evaluation	From “forecast only” to explicit backtesting and error metrics	The original proposal emphasized producing forward forecasts but did not specify a concrete evaluation strategy. The final app includes a backtest routine that holds out the last several weeks as a test window, re fits the selected model on earlier data, and reports MAE, RMSE, and MAPE, as well as MAE as a percentage of the	Without explicit evaluation, there is a risk that users over trust forecast curves that may not perform well in practice. Adding backtesting and simple, interpretable error metrics helps managers understand how reliable each model has been on recent data and encourages cautious, evidence informed use of forecasts. This also aligns with best practices in time series forecasting for healthcare operations.

Change ID	What changed	Details of the change	Justification / reasons
		average weekly exposure level.	
g) Stretch work: from generic “extra ML & AI” to a scoped AI Planning Assistant (PAi)	From open ended ML/AI experiments to a concrete AI recommendation layer on top of the forecast	The original stretch goals mentioned trying additional supervised ML models and an AI API for “weekly risk briefs” in general terms. In the final project, this has been narrowed into a specific, implemented feature: the AI Planning Assistant (PAi), which uses Google’s Gemini API to generate 4–6 sentence operational recommendations based on the recent history, chosen model, forecast horizon, and backtest metrics.	Early experiments showed that building many extra ML models would dilute effort away from the core forecasting problem. At the same time, there is clear value in helping non technical managers interpret the results. Implementing PAi as a focused AI layer on top of a single forecasting workflow delivers tangible AI functionality while keeping the scope manageable and directly tied to the app’s outputs.
h) Platform / tooling	From Databricks centric pipelines to SQL Server + CSV + Streamlit	The initial plan referenced Databricks Community Edition and large scale Spark processing. Due to storage and quota constraints, and the size/complexity of the NEMSIS research data, the workflow shifted to Microsoft SQL Server and SQL Server Management Studio for heavy joins and filtering, exporting a compact CSV of exposure events that the Streamlit app can load efficiently. The final front end is a Streamlit web app instead of a more complex multi service deployment.	Databricks Community Edition’s storage limits and friction around managing very large multi table datasets made it an awkward fit for this project. SQL Server was a better match for relational joins on large tables, and CSV export simplified the runtime environment. Streamlit provided a fast way to build an interactive, reproducible forecasting interface without separate frontend and backend stacks. This combination was more realistic to implement within the course timeline.
i) Hypotheses	From many operational hypotheses to a	The proposal outlined several hypotheses about workload, staffing,	Many of the original hypotheses require variables (detailed staffing, shift length, culture

Change ID	What changed	Details of the change	Justification / reasons
	small set focused on temporal patterns and operational usefulness	ergonomics, and culture. The final project instead focuses on a smaller set of testable questions: whether weekly exposure counts show meaningful temporal structure (trend/seasonality), whether simple time series models can forecast short term exposure counts with acceptable error, and whether presenting these forecasts plus AI generated recommendations in a dashboard could plausibly support proactive planning.	measures) that are not available or are too sparse in the underlying data. Narrowing the hypothesis set to what can actually be observed and evaluated keeps the project empirically grounded, while still connecting back to the original motivation of helping EMS services anticipate and manage exposure risk.

Project Completion Timeline

Overview

From the beginning of term to the final submission, the project progressed through data understanding and pipeline design, exposure-focused data engineering, time-series model development with backtesting, and Streamlit app implementation, with the AI Planning Assistant (PAi) added in the final phase. Compared with the earlier plan, the main changes were: concentrating on the 2024 exposure-flagged cohort only, shifting heavy preprocessing into SQL Server and CSV export, narrowing the modeling strategy to a small set of interpretable time-series models, and fully implementing forecast-accuracy checks and PAi recommendations instead of a broad comparison of generic ML models.

Phase Breakdown & Milestones

Phase	Main activities	Weeks (overall)	Key milestones & deliverables	Status
1. Setup, learning & exploratory data analysis	Environment setup; initial NEMSIS exploration; feasibility checks	Weeks 1–2 (plus light touches)	Local Python/Jupyter and Streamlit environment configured; initial NEMSIS sample loaded; basic	Completed

Phase	Main activities	Weeks (overall)	Key milestones & deliverables	Status
		in Week 3)	exposure distributions and time-of-day plots created; preliminary notes on data limitations and feasible outcomes.	
2. Data cleaning & pipeline foundation	Initial cleaning; schema understanding; early joins	Weeks 2–3	Basic cleaning on January and early-2024 data; first ETL scripts drafted; initial handling of missing/Not Applicable values; tested joins between incident and exposure elements; documented issues with hospital outcome completeness.	Completed
3. Exposure-focused data prep (final dataset)	Build the 2024 exposure cohort; weekly aggregation; CSV export	Weeks 3–6	Assembled full 2024 exposure-flagged cohort; confirmed usable time field for aggregation; implemented SQL/ETL to count exposure events per day; resampled to Monday-based weekly series; exported a compact CSV (PCR_Exposure_Final.csv) for use by the app.	Completed
4. Time-series model development & accuracy checks	Baselines and time-series models; backtest routine	Weeks 4–7	Implemented naive and moving-average baselines, simple exponential smoothing, Holt, Holt–Winters (with checks for minimum history), SARIMA, and a small experimental random-forest model; added a backtest function that holds out recent weeks and computes MAE, RMSE, MAPE, and MAE as a percentage of average weekly exposures; selected a default baseline and finalized how metrics are displayed.	Completed

Phase	Main activities	Weeks (overall)	Key milestones & deliverables	Status
5. Streamlit app core (EMS Forecaster)	Core UI, controls, charts, and tables	Weeks 5–8	Built a Streamlit app that loads the weekly series from CSV; added sidebar controls for model selection, options (events, weather, AI recommendation, confidence band), and “Run forecast” trigger; implemented main metrics (last historical week, model selected, horizon); created historical vs forecast line chart; added a forecast table with point forecasts and simple bounds.	Completed (core functionality)
6. Context enrichment and experimental features	Weather, events, and experimental ML	Weeks 6–8	Integrated optional weather enrichment via the Open-Meteo API and optional major-events counts via the Ticketmaster API, with safeguards when data are unavailable; wired these context features into the SARIMA pipeline; implemented the “original vs updated forecast” overlay as an illustrative feature; exposed the experimental “ML (Experimental)” model in the UI with clear labeling.	Completed (as optional features)
7. AI Planning Assistant (PAi) and narrative explanations	AI recommendation layer; manager-facing copy	Weeks 7–9	Implemented a narrative “Interpretation (for managers)” block that summarizes expected exposures over the next four weeks and, when weather is enabled, describes historical correlations with rain; integrated Google Gemini via an API key to generate 4–6 sentence recommendations that incorporate the forecast	Completed

Phase	Main activities	Weeks (overall)	Key milestones & deliverables	Status
			summary and backtest metrics; added a toggle for “Generate AI manager recommendation.”	
8. Final polish, documentation, and testing	UI refinements; code cleanup; report	Weeks 8–10	Cleaned and commented the Python app code; confirmed CSV path handling and caching; checked that all models run without errors for typical horizons; verified backtest metrics for several models; refined UI text and labels for non-technical users; finalized the written report sections (introduction, summary, changes to proposal, timeline, implemented features).	Completed

Gantt Chart

Activities	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9
Setup and EDA									
Databricks Trial									
AI Integration Plan									
Data Cleaning (incl. joins, exposure cohort)									
Feature Engineering / Weekly Aggregation									
Time-series model development (baselines, ETS, SARIMA)									
Backtesting and accuracy metrics (MAE, RMSE, MAPE)									
Model Selection / Signal Assessment									
AI-Driven Reporting (text recommendations)									
Dashboard Development & AI (Streamlit app)									
Final Polish / Buffer									

Implemented Features

This section summarizes the main features implemented in the EMS Work-Related Injury Forecast app, covering the data pipeline, forecasting logic, evaluation, user interface, and AI Planning Assistant. Each feature is described in terms of what it does, how it was designed and implemented, and the specific contribution it makes to the overall system. Several midterm prototype features (early rule-based text summaries and partial ML experiments) were consolidated into the final forecasting core and PAi components, so the final report groups them into five integrated features rather than the original seven.

Implemented feature	Description (what it does)	Design & implementation details	Contribution
1. Exposure-focused data pipeline	Builds a 2024 EMS exposure cohort and prepares it as a clean weekly series for forecasting.	2024 exposure data are imported into SQL Server and filtered to work-related exposure events, handling Not Applicable/Not Recorded codes. A compact CSV (PCR_Exposure_Final.csv) of exposure records is exported. In the app, <code>load_weekly_exposures_from_fact()</code> reads the CSV in chunks, parses timestamp strings, aggregates counts per day, and resamples to a Monday-based weekly time series used by all forecasting functions.	Designed the SQL filtering logic and exposure cohort, implemented the CSV-based loading/aggregation in Python, and added basic debug info (row counts, date ranges) to verify that the weekly series is complete and consistent.
2. Weekly exposure time series & forecasting core	Produces a validated weekly time series and several short-term forecasting options that EMS managers can select.	The pipeline constructs a single weekly exposure series (one point per week) from the cleaned cohort. Multiple forecasting functions are implemented: naive baseline, moving average, simple exponential smoothing, Holt, Holt-Winters (with history checks), SARIMA with optional exogenous weather/events, and an “ML (Experimental)” random-forest forecaster. All models return <code>week_start</code> , <code>yhat</code> , <code>yhat_lower</code> , and <code>yhat_upper</code> , and the “Prophet (default)” label currently uses a simple baseline forecast.	Implemented the forecasting functions, chose reasonable default settings (e.g., seasonal period for Holt-Winters, SARIMA orders), and exposed each model through the unified interface used by the app’s main control flow.

Implemented feature	Description (what it does)	Design & implementation details	Contribution
3. Backtesting and forecast-accuracy metrics	Evaluates how well each forecasting method performs on recent data and shows simple accuracy metrics with the forecast.	A <code>backtest_and_score()</code> routine holds out the last horizon weeks as a test window, fits the chosen model on earlier history, and forecasts the held-out period. It computes MAE, RMSE, MAPE, and MAE as a percentage of the average weekly exposure level. These metrics are displayed in a “Backtest accuracy” section (horizon, points, MAE, RMSE) and are also used to generate plain-language reliability text and to inform the AI Planning Assistant’s prompt.	Designed the backtesting approach, coded the scoring function, and integrated the metrics into both the dashboard (metric tiles) and the explanatory text about forecast reliability for non-technical users.
4. Streamlit dashboard & forecast visualization	Provides an interactive web interface where EMS managers can choose models/options and see historical exposures, forecasts, and tables in one place.	The Streamlit layout uses a sidebar for model choice, forecast horizon slider, and checkboxes (weather effects, major events, confidence band, original vs updated forecast, AI recommendation). The main panel shows key summary metrics (last historical week, model selected, horizon), an interactive line chart overlaying historical weekly exposures and the selected forecast, and a read-only forecast table with week, point forecast, and bounds. An optional “Original vs updated forecast” chart illustrates how forecasts can change as new data arrive.	Designed the UI flow, built the Streamlit components (sidebar, metrics, charts, tables), and wired them to the forecasting/backtesting functions so that changing any option re-runs the pipeline and refreshes the visuals.
5. AI Planning Assistant (PAi) & narratives	Generates plain-language explanations and recommendations that turn the forecasts and accuracy metrics into operational guidance for EMS managers.	Adds an “Interpretation (for managers)” text block summarizing the next-4-week expected average exposures and range, and, when enabled, commenting on historical relationships with rain. The AI Planning Assistant (PAi) uses Google’s Gemini API: <code>generate_recommendation()</code> builds a structured prompt containing a	Defined the PAi concept and prompt, implemented the Gemini call, and integrated the generated recommendation into the dashboard so managers can see both numeric forecasts and a concise, non-technical

Implemented feature	Description (what it does)	Design & implementation details	Contribution
		concise history/forecast summary, the selected model and horizon, and backtest metrics, and requests a 4–6 sentence recommendation focused on staffing, training, and PPE planning. A checkbox controls whether PAi is run.	action-oriented summary.

Reflections

AI Use Section

AI Tool Name	Version Account Type	Specific Feature for which the AI Tool was Used	Value Addition (What value did you add over and above what AI did for you?)
Perplexity	Education Pro	Asked AI to surface literature that would support framing the problem that demonstrates occupational hazard encountered EMS personnel	Having done a coop role in VCH, I learned that work-related injuries are handled by WorkSafeBC so I also asked AI to check for any statistics coming from the organization, this is on top of the 2 it produced on CUPE and JEMS
Perplexity	Education Pro	Asked AI to confirm if there's any AI or ML related studies or projects published using the NEMSIS data in order to check uniqueness of my study	Gave input to AI about the focus of my studies which is really about forecasting work-related injuries for emergency personnels so that it serves as comparison for other studies that may have been done
Perplexity	Education Pro	Asked AI to evaluate my timeline if reasonable and for any crucial activities I may have missed	Provide the milestones I crafted as well as the activities envisioned so that AI provide further input on anything I missed

AI Tool Name	Version Account Type	Specific Feature for which the AI Tool was Used	Value Addition (What value did you add over and above what AI did for you?)
Perplexity	Education Pro	Asked AI to help build and code the Streamlit EMS forecasting app.	Provided the requirements from selecting models like SARIMAR or prophet, week duration, special features like taking into account concerts and more as well as driving how the main content look like with bargraphs and recommendation.
Perplexity	Education Pro	Asked AI to review SQL codes particularly in troubleshooting the time format of the hospital admit time.	Discovered that with my SQL codes, no data is being filtered out so investigated why and found that time format is somehow messing up the filter so I asked AI 3 scenarios on the format.
Perplexity	Education Pro	Asked AI to refine the video script I created, improve the flow and make it more concise so that it's at least 6 minutes long.	Created the outline and initial video script, discussing the major contents I would like to be present in the video then feed it to AI for further improvement.
Perplexity	Education Pro	Asked AI to provide a crash tutorial on Streamlit – app to use and how it works	Provided requirements to build an app and to have dashboard. AI is able to narrow down to Streamlit as a platform that's able to meet my requirements.
Perplexity	Education Pro	Asked AI to provide options for apps that can process large dataset aside from databrick, given that the app is only able to handle 5-10gb data.	Provided the context that I would like to go with Databricks but given its limitations, I would like to explore more app.
Perplexity	Education Pro	Asked AI to layout a couple of approaches for me in terms of joining large dataset in Jupyter Notebook.	Provided context that I'm able to use SQL to narrow down first dataset but given I'm joining to other large dataset, Jupyter Notebook code kept on crashing.
Perplexity	Education Pro	Asked AI to build data connection SQL codes and incorporate into the app	Provided context and initial codes built in order for AI to enable the connection of the app to the SQL data

AI Tool Name	Version Account Type	Specific Feature for which the AI Tool was Used	Value Addition (What value did you add over and above what AI did for you?)
Chagpt	Free Account	Asked AI about how to properly store confidential credentials so that it's not uploaded in github	Provided situation where I have an app code that connects to SQL and would need to hide the credentials
Perplexity	Education	Asked AI to build actual forecasting models and incorporate into the app	Listed the models to be used and specified where to incorporate into the app
Perplexity	Education	Incorporate additional forecasting models into the app	To ensure robustness of forecasting models, prompted AI to add 2 more models.
Claude	Free Account	Build code to use LLM to generate recommendation	Gave specific input to refer to the forecast results and take those as input to generate recommendation output.
Perplexity	Education Pro	Asked AI to generate a separate CSS script dedicated to styling the app sidebar.	I initially built and embedded the CSS directly within the main Python file, then integrated the AI-generated version for cleaner structure and maintainability.
Canva AI	Free Account	Asked AI to design a logo concept for the EMS forecasting app.	I defined the color palette, specifying a teal and cyan theme inspired by diagnostic and healthcare color schemes, then customized the AI-generated version to match the app's interface.
ChatGPT	Free Account	Asked AI to troubleshoot visibility issues with the forecast table in Streamlit.	I iteratively tested AI's CSS suggestions, manually edited the stylesheet, and when the issue persisted, I reconfigured the config.toml file myself, which resolved the problem before requesting a cleaner CSS script from AI for future reference.
Perplexity	Education Pro	Asked AI to generate a PowerShell script to analyze previous-year text files, including checking headers and validating basic data structure.	Reviewed and tested the AI-generated PowerShell script against the actual datasets, adjusted header mappings and file paths, and refined the logic to handle data anomalies and

AI Tool Name	Version Account Type	Specific Feature for which the AI Tool was Used	Value Addition (What value did you add over and above what AI did for you?)
			missing fields specific to the project files.
ChatGPT	Free Account	Asked AI to generate SQL scripts to import text-based data files into the database tables used for exposure and event records.	Modified the AI-generated SQL scripts to match the project schema and naming conventions, added error handling, and implemented custom batch-completion messages so I could monitor progress and confirm that all files were successfully imported.
Perplexity	Education	Asked AI to build a code for accuracy check and incorporate into the app	Use MAE and RMSE to check accuracy of forecasting models
Perplexity	Education	Asked AI to modify the existing comparison graph and incorporate dropdown options for the 2 models.	Use model 1 vs model 2 for comparison

Work Log

Week	Date	Number of Hours	Description of Work Done
1	18-Jan-26	0.5	Meeting with Priya, initial discussion of topics
	21-Jan-26	2	NEMSIS Data initial proposal
	23-Jan-26	0.5	Meeting with Solaris, Tony Tsui
	24-Jan-26	1	Initial works on the final proposal, intro and proposed research project
2	25-Jan-26	2	Align Intro and Proposed Research Project on the final proposal – Riipen link finally given
	26-Jan-26	4	Final works on the final proposal – Estimate and update timelines, build contract, update references, update AI usage table and prompts and perform final review and revisions
	30-Jan-26	2	Review of NEMSIS website, submission of data request for the project, communication of updates to Tony

Week	Date	Number of Hours	Description of Work Done
			Take Datacamp course on Introduction to Databricks, Setup a community edition Databricks account,
	31-Jan-26	2.5	Review data sent by NEMSIS, extract data (about 2 hrs. processing to extract 200GB data), build initial python code to review data
3	01-Feb-26	2	Update project plan to Agile format, reconsider performing a subset of data performing activities of the whole project in a shorter span of time, create ems eda notebook
	03-Feb-26	2.5	Review data dictionary form NEMSIS and study the headers in the raw dataset. Align equivalent mapping in the data dictionary to get more meaning.
	05-Feb-26	2.5	Provide updates to Riipen stakeholder, further revise project plan. Update ems eda notebook
	06-Feb-26	1.5	Provide updates to Priya, consolidate and review all generated files including notebook, upload to github
	07-Feb-26	1.5	Build end-to-end notebook pipeline from EDA to initial model application and evaluation
4	08-Feb-26	2.5	Update notebook pipeline, troubleshoot time data format, review next steps for the succeeding week.
	10-Feb-26	2	Filter Jan2024 from PCR_Events table using MS SQL SM.
	12-Feb-26	2.5	Analyze Exposure data. Join with PCR Events – results: Null Values for eOther for Jan2024.
	14-Feb-26	3	Review count of YES/No records for Exposure data. Check hospital admit time of Yes data – results: Not Applicable.
5	15-Feb-26	2.5	Revisit hypotheses and pivot. Analyze FACTPCR dataset. Focus on 20K records, eg weekly exposure forecast using SARIMA or Prophet.
	16-Feb-26	2	Design dashboard for forecasting app. Ask forecaster friends some features and functions that are ideal for forecasting app.
	17-Feb-26	2.5	Initiate and code Streamlit app. Include sidebar settings and main context for dashboard.
	18-Feb-26	2	Reach out to NEMSIS report for additional provided injury data –got response that they don't have the data readily available. Build script for video. Finalize Streamlit app.
	19-Feb-26	3	Shoot and edit video
	20-Feb-26	3.5	Finalize video – add in slides, Initiate midterm report, update readme file
	21-Feb-26	2	Review and polish midterm checklists. Update AI Usage table and List of Prompts table. Upload video in youtube.
6	22-Feb-26	1	Finalize and upload midterm report to Blackboard (initial submission)
	23-Feb-26	2.5	Performed more works today using PCRTIME data to extract historical daily exposure data as time series to forecast using 7-day

Week	Date	Number of Hours	Description of Work Done
			moving average which current serves as best model compared to naïve and exponential smoothing.
	24-Feb-26	1	Preparation of presentation to employer and planning of next steps
	25-Feb-26	1	Employer session on project updates
	26-Feb-26	2.5	Initial build of incorporating data pre-processing in the app, shift from fake data to real data
	27-Feb-26	1	Midterm check-in. Continue incorporating SQL code connection in the app
	28-Feb-26	2	Incorporation of forecasting models into the app
7	01-Mar-26	1.5	App testing of data connection and the forecasting models, capture of issues
	02-Mar-26	1	Progress Report 2 update and submission.
	03-Mar-26	2	Include additional forecast models for seasonality – Holt & Holt-Winters. Include error handling for Holt-Winters if available data is only 1 year.
	04-Mar-26	1	Request of additional data, cover more years from 2020-2023
	05-Mar-26	2	Sending of weekly updates to client.
	06-Mar-26	2.5	Week 7 check-in, review data given and follow up additional needed data.
	07-Mar-26	2	Initial code works on connecting to AI API to generate recommendation section.
8	08-Mar-26	2	Review data given and follow up FACTPCRTIME data since it still not available in the 2 nd batch of files provided. Review requirements for hosting in Streamlit.
	09-Mar-26	1	Progress Report 3 update and submission.
	10-Mar-26	2	Added initial weather effect to the forecast and update the weather function. Finalize AI recommendation
	11-Mar-26	1	Initial analysis on the previous year data. Extract provided data. Explore data content
	12-Mar-26	1.5	Make AI recommendation optional, troubleshoot and test, update session with employer.
	13-Mar-26	1	Start redesigning sidebar, update sidebar background and font colors, short check-in with Priya
	14-Mar-26	2.5	Add app logo in the sidebar, separate CSS style, fix dropdown issues
9	15-Mar-26	2.5	Troubleshoot forecast table being invisible, troubleshoot css not surfacing needed style. Work on initial updates of Report 4.
	16-Mar-26	1.5	Finalize updating report 4 and upload to blackboard.
	17-Mar-26	2	Use ticket master and build event calendar API and incorporate to the code.

Week	Date	Number of Hours	Description of Work Done
	18-Mar-26	1	Continue analysing the previous year data. Build PowerShell script to confirm headers of data
	19-Mar-26	2.5	Fix sidebar dropdown issues. Update session with employer. Employer requested to add model accuracy check. Potential to attend EMS expo latter part of this year.
	20-Mar-26	2	Build SQL codes to import txt data -FACTPCRWorkRelatedExposure
	21-Mar-26	2	Populate SQL PCREvents and exposure tables
10	22-Mar-26	1	Sent follow up to NEMSIS support on the content of eTimes03 since content are not timestamps.
	23-Mar-26	1	Update and submit Progress Report 5. Provide screenshots of data to NEMSIS support.
	24-Mar-26	2	Add backtesting of forecast models
	25-Mar-26	1.5	Testing of deployment in Streamlit
	26-Mar-26	1	Update session with employer
	27-Mar-26	1	Check-in session with Priya
	01-Apr-26	2.5	Modification of forecast settings. Include dropdown of models and AI in the main board
11	02-Apr-26	1	Final closing session with employer
	03-Apr-26	1.5	Creation of user guide.
	04-Apr-26	1.5	Initial update of the Final Project report
	05-Apr-26	2	Initial creation of the presentation deck
	07-Apr-26	2.5	Update of ReadMe with the Installation guide. Review of user guide and presentation deck
	08-Apr-26	3.5	Final update of Final Project Report. Upload and submission to Blackboard

Concluding Remarks

From a data perspective, both 2023 and 2024 show a consistent seasonal pattern: injury-related exposures tend to peak in January and decline toward the end of the year. While the 2023 data still needs additional filtering and cleaning, the broad downward trend over the year is visible even in its current, somewhat inflated form. This suggests that the seasonality signal is strong enough to survive some data quality issues, but it also highlights the value of continuing to improve data hygiene in future iterations.

On the forecasting side, the simpler models (naive, moving average, and Prophet) produce relatively flat, straight-line projections, whereas exponential smoothing and Holt's method introduce more structure. Exponential smoothing tends to forecast higher exposure levels, and Holt's model even shows a modest upward trend. Back-testing tells a clearer story: exponential smoothing delivers a mean absolute error of about 31 and an RMSE around 37, while most other baseline models sit closer to 120–130. Holt performs poorly on this dataset (MAE ~221, RMSE ~258), and SARIMA lands close to exponential smoothing (MAE ~32, RMSE ~41). This combination of results supports using exponential smoothing or SARIMA as the primary operational models, with the others as sanity checks rather than decision drivers.

What is particularly interesting is how the AI-generated operational recommendation lines up with these quantitative results. It translates an average forecast of roughly 272 weekly exposures (versus about 193 recently) into clear staffing and supply guidance, and it explicitly bakes in the typical error range (around 32–41 exposures per week) to frame a practical planning band. In other words, the AI layer is not just producing text; it is coherently aggregating the behavior of different models and their back-testing performance into recommendations that are numerically grounded and operationally sensible.

Stepping back, this project shows how even with imperfect data and a short build timeline, it is possible to move from raw exposure records to a working pipeline: a cleaned and aggregated weekly signal, multiple forecasting models, quantitative back-testing, and an AI assistant that turns uncertainty into concrete, defensible staffing and logistics decisions. With more time, improving 2023 data quality, extending the forecast horizon, and embedding the tool more tightly into day-to-day EMS workflows would be natural next steps.

Finally, I want to acknowledge the people who made this work possible. Thank you to Tony Tsui for the opportunity to work on this project and for the trust to explore a new way of supporting EMS operations. I am grateful to Priya, my professor, for the time, effort, and guidance invested in getting this project off the ground and keeping it on track. I also want to thank VCH: without the Coop experience gained there, I would not have the appreciation or drive to take on projects like this. And, most importantly, I am deeply grateful to my wife and twins, who are my inspiration every day and the reason I keep pushing to build things that make a real-world difference.

References

Journals and Articles

- Alhazmi, F., Alshamrani, A., Nazer, R., Alshehri, A., & Almutairi, M. (2024). Prevalence of work-related musculoskeletal disorders and associated factors among emergency medical services personnel in Jordan. *BMC Musculoskeletal Disorders*, 25, Article 344.
- Hammad, M., Al Nawaiseh, N., Al-Hourani, H., & Al-Dwairi, A. (2025). Prevalence and associated factors of work-related musculoskeletal disorders among emergency medical service workers. *Scientific Reports*, 15, Article 12345.
- Marsh, E., Xia, T., & Collie, A. (2025). Profiling paramedic job tasks, injuries, and physical fitness. *Applied Ergonomics*, 114, 104153.
- Maguire, B. J., O'Neill, B. J., & O'Meara, P. (2023). Occupational injuries and illnesses among paramedicine clinicians in the United States. *Journal of Occupational and Environmental Medicine*, 65(10), 889–897.
- Qian, Z., Rojas, E., & Selevan, J. (2016). Forecasting emergency department hourly occupancy using time series models. *American Journal of Emergency Medicine*, 34(9), 1738–1744.
- Reichard, A. A., Marsh, S. M., & Moore, P. H. (2017). Occupational injuries and exposures among emergency medical services workers. *Prehospital Emergency Care*, 21(4), 420–431.
- Tarisyai, K., Silva, D., & Ferreira, J. (2024). Enhanced forecasting of emergency department patient arrivals using SARIMAX and machine learning approaches. *Scientific Reports*, 14, Article 23456.
- Shahidian, N., et al. (2025). Short-term forecasting of emergency medical services demand using machine learning techniques. *Omega*.
- Vieira, S., Cardoso, T., & Ferreira, J. (2025). Forecasting emergency room patient volumes using XGBoost. *Healthcare Analytics*, 6, 100262.
- Vieira, S., Cardoso, T., & Ferreira, J. (2023). Forecasting emergency department visits in the reference hospital of a developing country. *arXiv Preprint arXiv:2602.12808*.

Web Reports / Government and Organizational Reports

- Defence Research and Development Canada. (2017). *Canadian paramedic health and wellness project* (DRDC-RDDC-2017-R099).
- Edmonton Metro EMS. (2025, June 18). *Disabling injury rate*.

National Emergency Medical Services Information System (NEMSIS). (2019). *eOther.06—Type of work-related injury, death or suspected exposure (Data Dictionary v3.4.0)*.

National Emergency Medical Services Information System (NEMSIS). (2024). *2024 NEMSIS public-release research dataset*.

National Emergency Medical Services Information System (NEMSIS). (2025). *National EMS Database: NEMSIS research data set user manual (RDS 350)*.

National Emergency Medical Services Information System (NEMSIS). (2025). *National EMS database: 2024 data report*.

National Highway Traffic Safety Administration. (2025). *Nationwide EMS incident data*.

WorkSafeBC. (2001). *Evaluation of paramedics' tasks and equipment to control the risk of musculoskeletal injury* (Finding Solutions Grant 99FS-14).

Theses / Dissertations

Jones, C. (2024). *Machine learning-based demand prediction in Dublin, Ireland* (Master's thesis, National College of Ireland).

Appendix A: Installation Guide

Installation Guide is available in the GitHub repository.

https://github.com/dondemici/W26_4495_S2_DundeeA/blob/main/README.md

Appendix B: User Guide

Installation Guide is available in the GitHub repository.

https://github.com/dondemici/W26_4495_S2_DundeeA/blob/main/DocumentsAndReports/EMS%20Forecaster%20App%20User%20Guide.pdf

Appendix C: List of Prompts

Prompts
Given my project and objective on analyzing EMS data and forecasting information to improve QA protocols to lessen work-related injuries of emergency personnel, find relevant literature that would support demonstrating the occupational hazards encountered by these workers.
When the literature was given, I further prompted if AI can give me stats by WorkSafeBC, having learned about the organization during my coop role time in VCH.
Given that I am doing a project on NEMSIS data, provide confirmation on any studies done in the past that is related to AI or ML so that I understand the significance and uniqueness of my study. My study focuses on forecasting work-related injuries for emergency personnels.
Review and evaluate my timeline of the project from EDA, cleaning, model evaluation and selection until dashboard reporting and check if reasonable given the 10-12.5 hrs I can allot weekly for a 2-month period. Mention any activities I may have missed to include.
Help me design and code a Streamlit app for an EMS exposure-risk forecasting dashboard where the user can choose SARIMA or Prophet, select the forecast horizon with a slider, optionally toggle scenario flags (concerts, political rallies), and then see historical plus forecasted weekly exposure counts with prediction intervals.
Review and debug my SQL code for joining NEMSIS tables on PcrKey and converting hospital admit time / PCR completion time fields into a consistent datetime format that I can use for weekly aggregation.
Refine the video script I created for my midterm presentation on EMS exposure forecasting—improve the flow, make it more concise, and keep the runtime around 6 minutes while preserving key technical details.
Provide a crash tutorial on building a Streamlit app for data science projects: how to structure the main script, add sidebar controls, display plots and tables, and run the app locally.
Suggest options for tools or platforms that can process large datasets besides the free Databricks edition, given that it can only handle around 5–10 GB, and explain pros/cons for my NEMSIS project.

Prompts
Lay out a couple of approaches for joining large relational datasets in Python/Jupyter (e.g., from NEMSIS): when to rely on pandas merges vs doing joins in SQL first, and how to manage memory efficiently.
Have a Python-based EMS forecasting app and I need to connect it to a SQL database instead of using fake data. Here is my current app code and data structure—please write the SQL connection code (including queries) and show me exactly how to integrate it into my app so it can read real EMS data.
Have an app that connects to a SQL database, and I'm pushing the code to a public GitHub repo. How can I properly store and hide confidential credentials (server, username, password, connection string) so they are not committed to GitHub, and what is the recommended way to load them safely at runtime?
Building an EMS demand forecasting app and want to integrate the following models (e.g., SARIMA, Prophet, Naïve, Moving Average and Exponential Smoothing). Here is my current app structure and data pipeline—please write the model training and prediction code and show where to insert it so the app can generate and display forecasts based on the selected model.
Add two additional forecasting models for time series seasonality — Holt's Linear Trend and Holt-Winters methods. Implement automatic selection of the best model based on AIC or RMSE. Include error handling for Holt-Winters if only one year of data is available
Use the forecast results as input data. Generate a recommendations section that summarizes key performance trends and suggests actions based on predicted values. Structure the output in a brief, client-friendly format suitable for report inclusion
Help me create a separate CSS script that styles only the sidebar section of my Streamlit app. It should include background color, font color, spacing, and hover effects. I currently have inline CSS in my Python file, but I want to move it to a dedicated stylesheet
Design a simple, professional logo concept for an EMS forecasting app? I'd like to use a cyan and teal color palette, inspired by healthcare or diagnostic themes. The design should look clean and modern.
My forecast table in Streamlit isn't showing even though the data is loading correctly. Can you help me troubleshoot what might be wrong with my CSS or Streamlit configuration? Please suggest debugging steps or how to adjust the config.toml file if necessary.
Generate a PowerShell script that reads multiple previous-year EMS text files, confirms that the headers match a given schema, and reports any missing or mismatched fields.
Write SQL scripts to import text-based EMS exposure and event files into SQL Server tables (e.g., PCREvents and Exposure), including basic error handling and a message indicating when each batch import is completed.
Would like to use MAE and RMSE to determine accuracy of forecast models, incorporate the section into the app.
Want to use model 1 vs model 2 to compare results of forecast, modify the existing comparison code to show the forecast differences.